Applied Deep Learning — Exam Study Guide

Compiled Q&A based on Prof. Dr. Lars Ackermann-Igl's slides

Contents

1 Part A — Threshold Logic Units (TLUs)

1.0.1 Q1. What is a Threshold Logic Unit (TLU) and where does it come from historically?

Answer: A TLU is the oldest form of an artificial neuron, introduced by McCulloch & Pitts (1943). It is a simple computational unit that produces **one output** depending on one or more inputs, by comparing a weighted sum of the inputs against a threshold θ.

Analogy: a controller that turns a light bulb on (output = 1) only once a light-sensor signal exceeds a defined threshold θ.

General structure:

- Inputs $\mathbf{x} = (x_1, \ldots, x_n)$
- Weights $\mathbf{w} = (w_1, \ldots, w_n)$ (importance of each input)
- Threshold θ
- Output y

1.0.2 Q2. How is the output of a TLU calculated?

Answer:

$$y = f(\mathbf{xw}) = \begin{cases} 1, & \text{if } \mathbf{xw} \geq \theta \\ 0, & \text{otherwise} \end{cases} \qquad \text{with } \mathbf{xw} = \sum_{i=1}^{n} w_i x_i$$

This f is called the **Heaviside step function (with offset)**. The unit fires (outputs 1) only if the weighted sum of inputs reaches or exceeds the threshold.

1.0.3 Q3. Why do TLUs need weights at all — why not just sum the raw inputs?

Answer: Weights encode the **relative importance/distinctiveness** of each input for the decision.

- Example 1 (stop-sign detection): the octagon shape is more discriminative/important than the red color, so it should get a higher weight.
- Example 2 (credit-card fraud detection): it is not obvious whether transaction amount or transaction location is more predictive — the *weight* is what expresses this importance, and it is exactly what training has to discover.

1.0.4 Q4. How can a TLU implement basic logical functions? Give an example.

Answer: A TLU computes a linear decision boundary, so it can represent any **linearly separable** Boolean function.

Conjunction $x_1 \wedge x_2$: weights $w_1 = 3, w_2 = 2$, threshold $\theta = 4$.

$$y = f(3x_1 + 2x_2), \quad \theta = 4$$

Only $x_1 = x_2 = 1 \Rightarrow 3 + 2 = 5 \geq 4 \Rightarrow y = 1$; all other combinations give $< 4 \Rightarrow y = 0$ — matches the AND truth table.

Implication $x_2 \to x_1$**:** weights $w_1 = 2, w_2 = -2$, threshold $\theta = -1$. Geometrically, each TLU corresponds to a **separating line/hyperplane** in input space, with the weight vector orthogonal to that line and the threshold determining its offset.

1.0.5 Q5. Why can't we just manually design the weights/threshold of a TLU in general?

Answer: Manual (geometric) construction only works for at most **2–3 inputs**, because beyond that we can no longer visualize/reason about the separating hyperplane. For real-world problems (many inputs), we need an **automated** way to find good weights and thresholds from data — i.e., training/learning.

1.0.6 Q6. Describe the general (manual/conceptual) training procedure for a TLU before introducing the Delta Rule.

Answer:

1. Start with **random** weights and threshold.
2. Given a dataset, calculate the **error** e the TLU produces (initially: absolute difference between expected output o and actual output y).
3. **Tweak** weights/threshold to reduce this error.
4. **Repeat** from step 2 until error is sufficiently small or vanishes.

1.0.7 Q7. Why is the simple "absolute difference" error not usable for optimization, and what is the fix?

Answer: Using the absolute difference as error produces an error surface made only of **flat plateaus**. On a plateau the gradient is locally zero/constant everywhere, so from any point we **cannot tell in which direction** to adjust w and θ to reduce the error — there's no usable "downhill" direction.

Fix: multiply the error by the **distance of the weighted sum from the threshold** $(\mathbf{xw} - \theta)$. This:

(i) considers *how far* the weighted sum is from the threshold, and
(ii) yields a meaningful gradient/direction in which to move w and θ to reduce error.

This idea is the conceptual root of the **Delta Rule**.

1.0.8 Q8. State the Delta Rule (Widrow–Hoff Procedure) for TLUs — original (threshold + weights) form.

Answer: Given input vector $\mathbf{x} = (x_1, \ldots, x_n)$ with desired output o and current TLU output y:

$$\theta^{(new)} = \theta^{(old)} + \Delta\theta, \quad \Delta\theta = -\eta(o - y)$$
$$\forall i \in \{1, \ldots, n\}: \quad w_i^{(new)} = w_i^{(old)} + \Delta w_i, \quad \Delta w_i = \eta(o - y)\, x_i$$

- η ("eta") = **learning rate**, controls the magnitude of the update.
- This procedure is called the **Delta Rule** or **Widrow–Hoff Procedure**.

1.0.9 Q9. How can the threshold θ be eliminated as a "special case" in the Delta Rule?

Answer: Rewrite the firing condition:

$$\sum_{i=1}^{n} w_i x_i \geq \theta \iff \sum_{i=1}^{n} w_i x_i - \theta \geq 0$$

Define $-\theta = w_0 x_0$ with a **constant input** $x_0 = 1$ and a special **bias weight** w_0. Then:

$$\sum_{i=0}^{n} w_i x_i \geq 0$$

Now there is only **one type of parameter** (weights), simplifying the learning rule — the threshold is "absorbed" into the weight vector as w_0 (with input always clamped to 1).

1.0.10 Q10. Give the generalized (single-case) Delta Rule and the corresponding output function.

Answer:

$$\forall i \in \{0, \ldots, n\}: \quad w_i^{(new)} = w_i^{(old)} + \Delta w_i, \quad \Delta w_i = \eta(o - y)x_i$$

with output computed as

$$y = f(\mathbf{xw}) = \begin{cases} 1, & \text{if } \mathbf{xw} \geq 0 \\ 0, & \text{otherwise} \end{cases}, \quad \mathbf{xw} = \sum_{i=0}^{n} w_i x_i$$

Note $x_0 = 1$ always, and w_0 plays the role of $-\theta$.

1.0.11 Q11. Worked Example — apply the Delta Rule by hand.

Given: Training sample S_1: $x_1 = 1, x_2 = 1, o = 1$. Initial weights: $w_0 = -5, w_1 = 2, w_2 = 1$. Learning rate $\eta = 1$.

Step 1 — Calculate output:

$$y = f(-5 \cdot 1 + 2 \cdot 1 + 1 \cdot 1) = f(-2) = 0$$

Step 2 — Calculate weight deltas: (using $\Delta w_i = \eta(o - y)x_i$, with $o - y = 1 - 0 = 1$)

$$\Delta w_0 = 1 \cdot 1 \cdot 1 = 1, \quad \Delta w_1 = 1 \cdot 1 \cdot 1 = 1, \quad \Delta w_2 = 1 \cdot 1 \cdot 1 = 1$$

Step 3 — Update weights:

$$w_0^{new} = -5 + 1 = -4, \quad w_1^{new} = 2 + 1 = 3, \quad w_2^{new} = 1 + 1 = 2$$

Exam tip: Always recompute y with old weights first (Step 1), get the sign/magnitude of $(o - y)$, then apply it uniformly across all weights including w_0 (with $x_0 = 1$).

1.0.12 Q12. What is linear separability? Define it formally.

Answer: Two sets of points in a Euclidean space are **linearly separable** if and only if there exists at least one point/line/plane/hyperplane (depending on dimensionality) such that all points of one set lie strictly on one side, and all points of the other set lie on the other side (or on the boundary itself). I.e., the sets can be separated by a **linear decision function**.

1.0.13 Q13. What is the fundamental limitation of a single TLU? Give examples of functions it can and cannot represent.

Answer: A **single TLU can only compute linearly separable functions**, because it draws exactly one separating hyperplane.

- **Linearly separable** (computable by one TLU): conjunction (AND), disjunction (OR), implication, NAND, NOR, etc.
- **Not linearly separable** (impossible for one TLU): **XOR**, and **bi-implication / XNOR** ($x_1 \leftrightarrow x_2$).

For bi-implication, the true points $(0,0)$ and $(1,1)$ and the false points $(1,0)$ and $(0,1)$ cannot be separated by any single straight line in 2D.

1.0.14 Q14. How do we overcome the linear-separability limitation of TLUs?

Answer: By building a **network of TLUs** (multiple layers). A single TLU can only realize one linear boundary, but combining several TLUs (e.g., computing intermediate functions y_1, y_2, then combining them with another TLU) lets the network carve out **non-convex / non-linearly-separable regions**.

Example: Bi-implication $x_1 \leftrightarrow x_2 \equiv (x_1 \rightarrow x_2) \wedge (x_2 \rightarrow x_1)$ is built from **three TLUs**: - TLU computing $y_1 = x_1 \rightarrow x_2$ - TLU computing $y_2 = x_2 \rightarrow x_1$ - TLU computing $y = y_1 \wedge y_2$ (final output)

This also explains why the number of *linearly separable* Boolean functions is much smaller than the *total* number of Boolean functions (2^{2^n}) for $n \geq 3$ inputs — networks are required to cover the rest.

1.0.15 Q15. Worked Example — Inference through a 2-layer network of TLUs.

Given network (bi-implication), with $x_1 = 1, x_2 = 0$:

- Hidden layer: $y_1 = f(1 \cdot 1 - 2 \cdot 1 + 2 \cdot 0) = f(-1) = 0$; $y_2 = f(1 \cdot 1 + 2 \cdot 1 - 2 \cdot 0) = f(3) = 1$
- Output layer: $y = f(-3 \cdot 1 + 2 \cdot 0 + 2 \cdot 1) = f(-1) = 0$

So $y = f(f(x))$ — this is a **two-layer (two-stage) computation**: the network's output is a function of a function ("composition"), which is exactly the structural idea behind deep networks.

Key open question raised: the Delta Rule tells us how to adjust the **output-layer** weights (we know the desired output), but **how do we train the hidden-layer weights**, since we don't have a "target" value for hidden neurons? *(This question is resolved later via backpropagation — see Part C.)*

2 Part B — From TLUs to Artificial Neurons & Networks

2.0.1 Q16. What changes when going from a TLU to a general "artificial neuron"?

Answer: Structurally identical (inputs, weights, weighted sum), but the **hard step function** f (threshold/Heaviside) is replaced by a more general, typically **differentiable activation function** f. So:

$$y = f(\mathbf{w} \cdot \mathbf{x}) = f\Big(\sum_{i=0}^{n} w_i x_i\Big)$$

This generalization is essential because it enables gradient-based learning (see Part C).

2.0.2 Q17. What is the "bias" and what role does it play?

Answer: The bias is the special weight w_0 paired with a constant input $x_0 = 1$ (the same trick used to fold θ into the weight vector for TLUs).

Role: it lets the neuron's activation function be **shifted** — i.e., approximate functions that do *not* pass through the origin $(0, 0)$.

Analogy with a line $f(x) = ax + c$: the **weight** = **slope** (a), the **bias** = **intercept** (c). Without a bias, the network could only fit functions forced through the origin.

2.0.3 Q18. List the four commonly used activation functions, their formulas, and their pros/cons.

Answer:

Function	Formula	Pros	Cons
Sigmoid	$f(x) = \dfrac{1}{1+e^{-x}}$	Smooth, outputs in $(0, 1)$	Vanishing-gradient problem; not centered around 0; computationally complex (exponential)
Tanh	$f(x) = \dfrac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	Same shape as sigmoid but **centered at 0**	Gradient problem still exists; even more complex than sigmoid
ReLU	$R(x) = \begin{cases} x & x \geq 0 \\ 0 & \text{else} \end{cases}$	Very cheap to compute; solves the vanishing-gradient problem	Can produce "dead neurons" (never activate again)
Leaky ReLU	$R(x) = \begin{cases} x & x \geq 0 \\ \alpha x & \text{else} \end{cases}$	Fixes the dead-neuron problem of ReLU; centered around 0	Gradient problem can still occur; more complex than ReLU

Side note from the slides: in practice, having *some* dead neurons is not necessarily harmful.

2.0.4 Q19. Formally define an Artificial Neural Network (ANN) as a graph.

Answer: An ANN is a directed graph $G = (U, C)$ where:

- U is a finite set of nodes (**units / neurons**).
- $C \subseteq U \times U$ is a finite set of directed edges (**connections**), each with an associated weight.
- The weight of the connection from the v-th neuron in layer l to the u-th neuron in layer $l+1$ is denoted $w_{uv}^{(l)}$.
- The output value of a neuron u is y_u.
- Neurons are grouped into **layers**; a layer's output is the vector of all its neurons' values.
- **Width** = dimensionality of the layers (number of neurons per layer).
- **Depth** = number of hidden layers.

2.0.5 Q20. Distinguish Single-Layer Perceptron (SLP) vs. Multi-Layer Perceptron (MLP).

Answer:

- **SLP:** inputs connect directly to a single output layer — no hidden layer. Equivalent to a (set of) TLU(s)/neurons in parallel. Can only solve linearly separable problems.
- **MLP:** one or more **hidden layers** between input and output. Composition of multiple non-linear transformations allows modeling of non-linearly-separable / arbitrarily complex functions (universal approximation, see Q21).

2.0.6 Q21. What is "Forward Propagation"? Give the general formula for an MLP with several layers.

Answer: Forward propagation is the process of computing the network's output by successively applying each layer's weights and activation function to the previous layer's output:

$$\vec{h}_1 = f(\vec{x} \cdot W_1), \quad \vec{h}_2 = f(\vec{h}_1 \cdot W_2), \quad \vec{h}_3 = f(\vec{h}_2 \cdot W_3), \quad \vec{y} = f(\vec{h}_3 \cdot W_4)$$

In general:

$$\vec{y} = f(\mathbf{xw}) = f^{(n)}\big(\dots f^{(2)}\big(f^{(1)}(\vec{x} \cdot W_1) \cdot W_2\big) \cdots \cdot W_n\big)$$

Implementation note: this is implemented as efficient **matrix multiplications** (crucial for GPU acceleration): $Z_H = XW_H + B_H$, $H = f(Z_H)$, and analogously for the output layer.

Important theoretical result: A neural network with a **single hidden layer** is already a **universal function approximator** — and increasing the number of neurons in that hidden layer improves the approximation quality.

2.0.7 Q22. Worked Example — Forward pass in an MLP with identity activation.

Given: $x = 0.8$; $w_{H1} = 0.2$, $w_{H2} = 0.1$; $w_{O1} = 0.5$, $w_{O2} = 0.3$, $w_{O3} = 0.2$, $w_{O4} = 0.7$; activation = identity ($f(x) = x$).

Hidden layer:

$$H_1 = f(x \cdot w_{H1}) = 0.8 \times 0.2 = 0.16$$

$$H_2 = f(x \cdot w_{H2}) = 0.8 \times 0.1 = 0.08$$

Output layer (cross-connections $O_1 = H_1 w_{O1} + H_2 w_{O2}$, $O_2 = H_1 w_{O3} + H_2 w_{O4}$):

$$O_1 = 0.16 \times 0.5 + 0.08 \times 0.3 = 0.08 + 0.024 = 0.104$$

$$O_2 = 0.16 \times 0.2 + 0.08 \times 0.7 = 0.032 + 0.056 = 0.088$$

(This is a standard style of exam question — practice plugging numbers through a small 2–2–2 MLP.)

3 Part C — Training ANNs: Gradient Descent & Backpropagation

3.0.1 Q23. Why can't the (single-layer) Delta Rule be used directly to train an MLP's hidden layers?

Answer: The Delta Rule needs the **desired/target output** o for the neuron being updated. For the **output layer**, training data directly gives us this target. But for **hidden layers**, we have **no ground truth** telling us what their "correct" output should have been. We need an algorithm that can decide how to *use* (and update) the hidden layers without explicit targets for them — this is exactly why training deep networks is called **"deep learning"**, and it requires the **Backpropagation algorithm**.

3.0.2 Q24. List the four general learning paradigms for ANNs.

Answer:

1. **Supervised learning** — for every input, the desired output is known (this is the course's focus).
2. **Unsupervised learning** — outputs unknown; the goal is to find structure in the inputs.
3. **Semi-supervised learning** — desired output known only for *some* inputs.
4. **Reinforcement learning** — outputs unknown; instead a reward function over inputs/actions must be discovered.

3.0.3 Q25. What is Gradient Descent? State its core update rule.

Answer: Gradient Descent is an iterative optimization method:

1. Start from a random parameter set $w = w_{init}$.
2. Repeatedly move w in the direction of **steepest descent** (negative gradient) of the error/cost function $J(w)$:
$$w := w - \eta \nabla J(w)$$
3. Repeat until convergence or a preset number of steps.

Intuitively: this is like a ball **rolling downhill** on the error surface until it settles in a valley (minimum). Practically, this means computing the partial derivative of the error with respect to **each individual weight**.

3.0.4 Q26. What does the sign/value of the gradient at a point tell us?

Answer:

- **Positive gradient:** increasing w increases the loss $\rightarrow$ should decrease w.
- **Negative gradient:** increasing w decreases the loss $\rightarrow$ should increase w.
- **Zero gradient:** locally, changing w slightly does not affect the loss (e.g., at/near a minimum or on a plateau).

3.0.5 Q27. What is the role of the learning rate η in gradient descent, and what happens if it's too small/large?

Answer: η scales the size of each update step.

- **Too small:** convergence is very slow (many tiny steps needed).
- **Too large:** the updates can overshoot the minimum, causing oscillation or divergence (bouncing back and forth across the valley, possibly never converging).

A commonly cited starting value (with normalized/scaled features) is $\eta = 0.01$.

3.0.6 Q28. Why can't gradient descent be applied directly to classic TLUs (step-function units)?

Answer: Gradient descent requires the function to be **differentiable**. The **Heaviside step function** has derivative 0 everywhere except at $x = 0$, where it is **undefined** (the "distributional derivative" is the Dirac delta function, which is not usable in practice either). With a derivative of (almost) zero everywhere, there is no gradient information to follow — hence the need to replace the step function with smooth, differentiable activation functions like sigmoid, tanh, or ReLU.

3.0.7 Q29. Outline the Backpropagation algorithm (high-level steps).

Answer:

1. **Initialize** all connection weights randomly.
2. **Forward pass:** compute the network output for a given input pattern.
3. **Compute the error** of the output (and its gradient) relative to the desired target.
4. **Backward pass:** propagate the error backwards through the network, using it (via gradient descent) to adjust all weights — including hidden-layer weights.
5. **Repeat** from step 2 until the error is below a threshold or a maximum number of iterations is reached.

Key insight: *Gradient descent is the method that updates the weights; backpropagation is the procedure that efficiently computes the errors/gradients needed for those updates* (via the chain rule).

3.0.8 Q30. Why does weight initialization matter, and what is a recommended range for tanh?

Answer: Random initialization is essentially choosing a random *starting point* on the error surface. The right scale/distribution matters for convergence speed and final accuracy, and depends on the activation function. For example, **tanh saturates** for large inputs (output $\approx \pm 1$, gradient ≈ 0), so very large weights make learning very hard ("stuck" gradients). Recommended initialization range for tanh (with normalized/standardized inputs): **[-4, 4]**.

3.0.9 Q31. What is the formula for the error/delta at the OUTPUT layer, and at a HIDDEN layer? Explain the difference.

Answer:

$$\delta_i^{(l)} = \begin{cases} f'(net_i)\,(o_i - y_i), & \text{if } i \text{ is in the output layer} \\ f'(net_i) \sum_k \left(\delta_k^{(l+1)} w_{ki}\right), & \text{otherwise (hidden layer)} \end{cases}$$

with $net_i = \sum_j w_{ij}\, o_j$ (the weighted input to neuron i), and for the sigmoid, $f'(x) = f(x)(1 - f(x))$.

Explanation:

- **Output neuron:** we directly know the target o_i, so the delta is simply the (derivative-scaled) difference between target and actual output.
- **Hidden neuron:** there's no direct target, so its delta is computed by **summing the deltas of all neurons in the next layer**, each weighted by the connection weight from this hidden neuron to that next-layer neuron. This is exactly how the error gets "propagated backwards" layer by layer.

3.0.10 Q32. State the generalized weight-update rule used in backpropagation (relation to the Delta Rule).

Answer:

$$w_{ij}^{(l)} = w_{ij}^{(l)} + \Delta w_{ij}^{(l)}, \qquad \Delta w_{ij}^{(l)} = \eta\, \delta_i^{(l)}\, o_j^{(l-1)}$$

where $o_j^{(l-1)}$ is the output of the **source neuron** feeding into i (or simply the input x_j, if $l = 1$).

This is a direct generalization of the original Delta Rule ($\Delta w_i = \eta(o - y)x_i$) — the difference $(o - y)$ is replaced by the more general $\delta_i^{(l)}$, which (for hidden layers) is computed recursively from the deltas of the following layer.

3.0.11 Q33. Worked Example — Backpropagation by hand (one full iteration).

Network: 2 inputs ($x_1 = 0.7, x_2 = 0.6$) + bias, 2 hidden neurons (h_1, h_2) + bias, 2 output neurons (o_1, o_2). Activation: sigmoid. Initial weights as given in the lecture (layer 1: $0.3, 0.8, 0.5$ / $-0.2, -0.6, 0.7$; layer 2: $0.2, 0.4, 0.3$ / $0.1, -0.4, 0.9$). Target output $o^{(2)} = (0.9, 0.2)$. $\eta = 1$.

Step 1 — Forward pass: - Net input to h_1, h_2: $(1.16, -0.2)$ → activations $(0.761, 0.45)$ - Net input to o_1, o_2: $(0.639, 0.2)$ → activations $(0.655, 0.55)$

Step 2 — Output error:

$$\delta^{(2)} = \begin{pmatrix} 0.9 \\ 0.2 \end{pmatrix} - \begin{pmatrix} 0.655 \\ 0.55 \end{pmatrix} = \begin{pmatrix} 0.245 \\ -0.35 \end{pmatrix}$$

Step 3 — Output-layer deltas (using $f'(x) = f(x)(1 - f(x))$):

$$\delta(o_1) = f'(0.639) \times 0.245 = 0.655(1 - 0.655)(0.245) \approx 0.055$$

$$\delta(o_2) = f'(0.2) \times (-0.35) \approx -0.087$$

Step 4 — Hidden-layer deltas (sum over next-layer deltas × connecting weights, then scale by $f'(net)$):

$$\delta(h_1) = f'(1.16)\big[\delta(o_1)\cdot 0.3 + \delta(o_2)\cdot 0.9\big] \approx -0.015$$

$$\delta(h_2) = f'(-0.2)\big[\delta(o_1)\cdot 0.4 + \delta(o_2)\cdot (-0.4)\big] \approx 0.010$$

Step 5 — Weight changes ($\Delta w_{ij} = \eta\,\delta_i\,o_j$):

$$\Delta w^{(2)} = \begin{pmatrix} 0.055 \\ -0.087 \end{pmatrix}\begin{pmatrix} 1 & 0.761 & 0.45 \end{pmatrix} = \begin{pmatrix} 0.055 & 0.042 & 0.025 \\ -0.087 & -0.066 & -0.039 \end{pmatrix}$$

$$\Delta w^{(1)} = \begin{pmatrix} 0.010 \\ -0.015 \end{pmatrix}\begin{pmatrix} 1 & 0.7 & 0.6 \end{pmatrix} = \begin{pmatrix} 0.010 & 0.007 & 0.006 \\ -0.015 & -0.011 & -0.009 \end{pmatrix}$$

Step 6 — Update weights, then go back to Step 1 for the next iteration.

Interesting observation noted in the lecture: the **relative weight change** is much larger for output-layer weights (≈0.84 on average) than for hidden-layer weights (≈0.027) in this example — hidden layers tend to learn **more slowly**, foreshadowing the vanishing-gradient problem.

3.0.12 Q34. How is the total/batch error of a network usually computed, and why not just sum raw differences?

Answer: For a single neuron u and sample s: $\delta_u^{(s)} = o_u^{(s)} - y_u^{(s)}$.

Simply **summing** these differences over all output neurons/samples can cause positive and negative errors to **cancel out**, hiding the true error magnitude. The fix is to **square** the error (basis of Mean Squared Error):

$$\delta_u^{(s)} = \big(o_u^{(l)} - y_u^{(l)}\big)^2$$

The total error over the training set S and output neurons U_{out}:

$$\delta = \sum_{s\in S}\sum_{u\in U_{out}} \big(o_u^{(l)} - y_u^{(l)}\big)^2$$

The evolution of this value over training iterations is called the **learning curve**, and training is typically stopped once it reaches an acceptably small, user-defined value.

4 Part D — Problems with Gradient Descent / Backpropagation

4.0.1 Q35. Explain the Vanishing Gradient problem: cause and mechanism.

Answer: Definition: Gradients become extremely small as they are propagated backward through many layers, causing **early layers to learn very slowly or stop learning** altogether.

Mechanism (chain rule): Because of the layered/nested structure $\vec{y} = f^{(n)}(\dots f^{(1)}(\vec{x}W_1)\dots W_n)$, the derivative w.r.t. an early-layer weight is a **product of many derivative terms** (chain rule):

$$\frac{\partial y}{\partial W_1} = \frac{\partial y}{\partial a^{(n)}} \cdot \frac{\partial a^{(n)}}{\partial a^{(n-1)}} \cdots \frac{\partial a^{(1)}}{\partial W_1}$$

Concrete cause: For the **sigmoid** activation, $0 \leq \sigma'(x) \leq \frac{1}{4}$. Multiplying many numbers ≤ 0.25 together (one per layer) makes the product shrink **exponentially** with network depth ($|\partial y/\partial W_1| \leq 0.25^n\, C$) — so the gradient converges towards 0 for deep networks.

Other cause: very small weight initializations (same multiplicative argument); low learning rates can mask/amplify the symptom.

Why ReLU helps: $f'(x) = 1$ for $x \geq 0$ and 0 otherwise — so along the "active" path the gradient is **not shrunk** as it passes backward (though it can cause dead neurons if $x < 0$ everywhere for a given neuron).

4.0.2 Q36. Explain the Exploding Gradient problem: cause and mechanism.

Answer: Definition: Gradients become **too large** during backpropagation, causing unstable training and possible divergence.

Causes: - Activation functions with derivative > 1 in deep networks — same chain-rule argument as vanishing gradients, but multiplying numbers > 1 makes the product grow exponentially instead of shrinking. - Bad (too large) weight initialization. - High learning rates can amplify/mask the problem.

4.0.3 Q37. Describe four geometric pathologies of the error surface that hinder gradient descent, and their typical remedies.

Answer:

Pathology	Description	Typical Remedy
(a) Suboptimal local minima	Gradient descent can get stuck in a local (non-global) minimum; the problem grows with the size/complexity of the error surface; no universal fix exists — in practice we accept "good enough" minima.	Adjust learning parameter

Pathology	Description	Typical Remedy
(b) Flat plateaus	On a flat region the gradient is nearly 0, so progress is extremely slow even though we haven't converged.	Add a **momentum term**
(c) Steep canyons	A sudden switch between very strong negative and positive gradients can cause **oscillation** back and forth.	Add a **momentum term**
(d) Overshooting good minima	On a steep slope, large gradient $\rightarrow$ large step $\rightarrow$ a good minimum can be skipped/left entirely.	Adjust learning parameter

---

4.0.4 Q38. What are overfitting and underfitting? How do we detect/prevent them?

Answer:

- **Overfitting:** the network essentially **memorizes** the training samples (achieves great training performance) but generalizes poorly — it gives wrong answers for new/unseen inputs of the same class. The learned decision boundary is overly tight around training points.
- **Underfitting:** the model is too simple / boundary too coarse to capture the true structure of the data (e.g., a straight line splitting points that need a non-linear boundary).
- **Well learned:** a decision boundary that generalizes correctly to unseen points of the same distribution.

Detection / prevention — train/test split: - **Training set:** used to adjust model parameters. - **Test (verification) set:** unseen data used to measure generalization. - Common split: **70% training / 30% verification** (random selection). - **Stopping criterion:** training can be considered "done" once the network performs well on **both** the training set and the verification set (good performance on training data alone is not sufficient evidence of success).

5 Part E — Practical Implementation (Keras) & Architectures

5.0.1 Q39. What are the typical preprocessing steps before training an MLP, and why is one-hot encoding needed?

Answer: Two main preprocessing steps:

1. **Label encoding:** ANNs cannot consume categorical labels directly. **One-hot encoding** converts each category into a binary vector, e.g.:

$$\{\text{Iris setosa, Iris virginica, Iris versicolor}\} \to \{[1,0,0],[0,1,0],[0,0,1]\}$$

2. **Train-test split:** divide data into a **training set** (used to adjust parameters) and a **test set** (unseen data used to measure generalization).

5.0.2 Q40. What does the Softmax activation function do, and where is it typically used?

Answer: Softmax converts a vector of raw output scores ("logits") into a **probability distribution** over all classes (values in $(0,1)$ that sum to 1):

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

It is the **typical last-layer activation in classification models** with more than 2 classes, since it lets us interpret the output as "confidence per class."

5.0.3 Q41. In Keras, why does a `Dense` layer have more parameters than just the number of weights you'd expect (e.g., 50 instead of 40, or 33 instead of 30)?

Answer: Because each Dense (fully connected) layer also includes a **bias term per neuron**, in addition to the input-weight connections. E.g., for an input dimensionality of 4 and 10 hidden neurons: $4 \times 10 = 40$ connection weights + **10 biases** = **50 parameters**. Likewise for 10→3: $10 \times 3 = 30$ weights + **3 biases** = **33 parameters**. This matches the slides' "50 weights ... 33 weights ... but why?" — answer: **don't forget the bias terms**.

5.0.4 Q42. What three things must be configured when compiling/training a Keras model, and what do `epochs` and `batch_size` mean?

Answer:

- **Optimizer:** tells the model *how* to update weights during training (e.g., `adam`).
- **Loss function:** measures the error between predicted and expected outputs (e.g., MSE for regression, categorical cross-entropy for classification).
- **Metrics:** human-readable performance indicators tracked during training (e.g., accuracy).

Additional training parameters: - **epochs:** how many times the model passes over the **entire** training dataset. - **batch_size:** how many samples the model processes **before each weight update**.

5.0.5 Q43. Name the main ANN architectures and which task category (regression, classification, time series, generation, clustering) each is typically used for.

Answer:

Task	Architectures
Regression	Single-layer Perceptron (SLP), Multi-layer Perceptron (MLP), Convolutional Neural Nets (CNN), Radial Basis Function Nets (RBF)
Time Series	Elman-Nets, Recurrent Neural Networks (RNN), LSTM, Gated Recurrent Units (GRU)
Generation / Sequence-to-Sequence	RNN, Generative Adversarial Networks (GAN), Autoencoders (AEN), Transformers
Classification	MLP, RNN, LSTM, CNN
Clustering	Self-Organizing Maps (SOM), Embeddings

(MLP and SLP, highlighted as the focus of these lectures, cover the **regression** and **classification** branches respectively at the basic level.)

6 Quick-Reference Formula Sheet

- **TLU output:** $y = f(\mathbf{xw})$, step function, $\mathbf{xw} = \sum_i w_i x_i \geq \theta$
- **Bias trick:** $-\theta = w_0 x_0$, with $x_0 = 1$, so $\mathbf{xw} = \sum_{i=0}^{n} w_i x_i \geq 0$
- **Delta Rule:** $\Delta w_i = \eta(o - y)x_i$
- **Neuron output (general):** $y = f(\mathbf{w} \cdot \mathbf{x})$, f = differentiable activation
- **Forward propagation:** $\vec{y} = f^{(n)}(\ldots f^{(1)}(\vec{x} \cdot W_1)\ldots W_n)$
- **Sigmoid:** $f(x) = \frac{1}{1+e^{-x}}$, $f'(x) = f(x)(1 - f(x))$, $f' \in [0, 0.25]$
- **ReLU:** $f(x) = \max(0, x)$, $f'(x) = 1$ if $x \geq 0$ else 0
- **Gradient descent:** $w := w - \eta \nabla J(w)$
- **Backprop output delta:** $\delta_i^{(l)} = f'(net_i)(o_i - y_i)$
- **Backprop hidden delta:** $\delta_i^{(l)} = f'(net_i) \sum_k \delta_k^{(l+1)} w_{ki}$
- **Backprop weight update:** $\Delta w_{ij}^{(l)} = \eta\, \delta_i^{(l)}\, o_j^{(l-1)}$
- **Squared error (per neuron/sample):** $(o_u - y_u)^2$